Programming 2

Tutorial 11

Activity 1

Goal: Understand that Java programs start with a single thread.

Task: Write a Java program that:

- Prints the name of the currently running thread.
- Pauses execution for 2 seconds.
- Prints a message after the pause.

Activity 2

Goal: Learn how to create and start a new thread using `Runnable`.

Task:

1. Create a class `PrintTask` that implements `Runnable`.
2. The `run()` method should:
    - Print `"Task started"`
    - Wait for 3 seconds
    - Print `"Task finished"`
3. In `main()`:
    - Start the above thread
    - Immediately print `"Main thread continues..."`
2. Explain the program's console output using your knowledge of Java Thread.

Activity 3

Goal: Observe the difference between blocking and multithreading.

Task: You are given a method:

```
public static void downloadFile(String name) {
    System.out.println("Downloading " + name);
    try {
        Thread.sleep(3000);
    } catch (InterruptedException e) {}
    System.out.println("Finished " + name);
}
```

1. Call this method twice sequentially in `main()`.
2. Measure total execution time.
3. Rewrite using 2 threads so both downloads happen concurrently.
4. Measure the new total time.

Hint: Use `System.currentTimeMillis()` to measure time.

Please explain:

- Why is the concurrent version faster?
- Does the output order stay the same in multiple runs of the program?

Activity 4

Goal: Ensure the main thread waits for worker threads.

Task: You are processing exam scores. The scores are split into two halves, and each half should be processed in parallel.

1. Create a class `SumTask` that implements `Runnable`:
    - Takes an `int[]` array
    - Computes the sum of the array
    - Stores the result in an instance variable
2. In `main()`:
    - Create an array of at least 10 integers (scores)
    - Split it into 2 smaller arrays
    - Create 2 threads to compute partial sums
    - Start both threads
3. After both threads finish:
    - Print the total sum

You must **wait for both threads** before computing the final result.

Activity 5

Goal: Practice independent threads running alongside user input.

Task: You are building a simple console-based chat simulator:

- Messages from "other users" appear randomly
- The main thread handles user input

Detailed Tasks:

1. Create a class `IncomingMessageTask` that implements `Runnable`:
    - Runs in a loop (e.g., 5–10 times)
    - Every 1–2 seconds, prints a random message like:

      ```
      "Alice: Hi!"
      "Bob: Are you there?"
      ```

2. In main():
    - Start the IncomingMessageTask in a new thread
    - Continuously allow the user to type messages:

      ```
      You: <input>
      ```

3. Both should run independently. Incoming messages should appear even while the user is typing.

Hint: Use `Thread.sleep()` with random intervals.

Submission

Submit a **zip** file containing all Java programs to this tutorial's submission box in the course website on FIT Portal.